

Q1. Following the 2-D DWT algorithm given in homework assignment 1, please conduct fixed point simulations to determine the DWT word length for the following items. Assume a floating point version IDWT is used in image reconstruction, the PSNR (peak signal to noise ratio) of the reconstructed image should be no less than 40dB. Please use as small word length as possible to achieve this goal. You need to indicate both the “word length” and the “weight” of LSB of each variable.

- The word length of the filter coefficients (Table 1), all coefficients should have the same word length. You can determine whether they will share the same weight or not. If they share the same weight, the effective word length of smaller coefficients will be reduced. However, there is no need of weight alignment when performing addition in filtering.

Analysis Filter Coefficients		
i	Lowpass Filter h_i	Highpass Filter g_i
0	0.852698679009	-0.788485616406
± 1	0.377402855613	0.418092273222
± 2	-0.110624404418	0.040689417609
± 3	-0.023849465020	-0.064538882629
± 4	0.037828455507	

- Determine the word lengths (and the weights) of the filter outputs at each level, i.e., level 1, 2 and 3 of DWT.
- Note 1:** For 2-D DWT, you need to perform 1-D DWTs horizontally first, followed by performing 1-D DWTs vertically using the results of horizontal DWTs. Therefore, the word length designs for horizontal and vertical DWTs can be different.
- Please summarize your final bit-true algorithm design by listing the word length and weight of each variable in 2-D DWT, and calculate the PSNR of reconstructed image.
- Note 2:** reconstructed image is obtained by performing fixed point forward 2-D DWT followed by performing floating point inverse 2-D DWT

(a) Determine the word length in DWT by fixed-point simulations:

Matlab code:

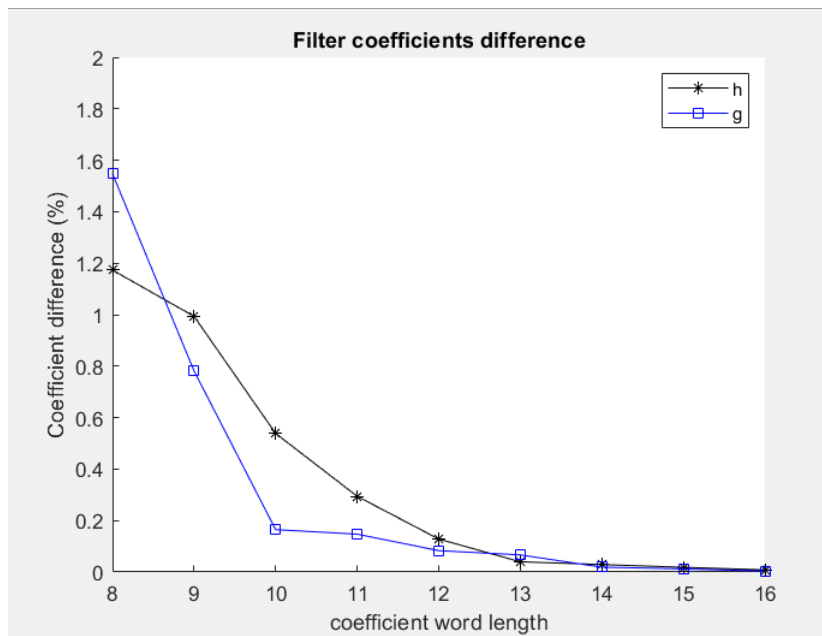
```
clc; clear; close all;
%% Choose word length
% filter coefficients
hFilter = [0.037828455507; -0.023849465020; -0.110624404418; 0.377402855613; 0.852698679009;
0.377402855613; -0.110624404418; -0.023849465020; 0.037828455507];
gFilter = [0; -0.064538882629; 0.040689417609; 0.418092273222; -0.788485616406; 0.418092273222;
0.040689417609; -0.064538882629; 0];
qFilter = [0; -0.064538882629; -0.040689417609; 0.418092273222; 0.788485616406; 0.418092273222; -
0.040689417609; -0.064538882629; 0];
```

```
pFilter = [-0.037828455507; -0.023849465020; 0.110624404418; 0.377402855613; -0.852698679009;
0.377402855613; 0.110624404418; -0.023849465020; -0.037828455507];
```

```
W = 16;
h_error = zeros(W, 1); % 16X1 zero array
g_error = zeros(W, 1);
for i = 8 : W
    h_fix = fi(hFilter, 1, i, i-1); % fixed-point(content,1 ,word_length, fraction_length)
    g_fix = fi(gFilter, 1, i, i-1);
    h_error(i) = sum(abs(hFilter - double(h_fix))) * 100; % difference percentage
    g_error(i) = sum(abs(gFilter - double(g_fix))) * 100;
end
```

```
figure(1)
hold on
plot(h_error,'k*','-');
plot(g_error,'bs','-');
legend('h', 'g');
xlabel('coefficient word length');
ylabel('Coefficient difference (%)');
xlim([8 W]);
ylim([0 2.0]);
title('Filter coefficients difference');
hold off
```

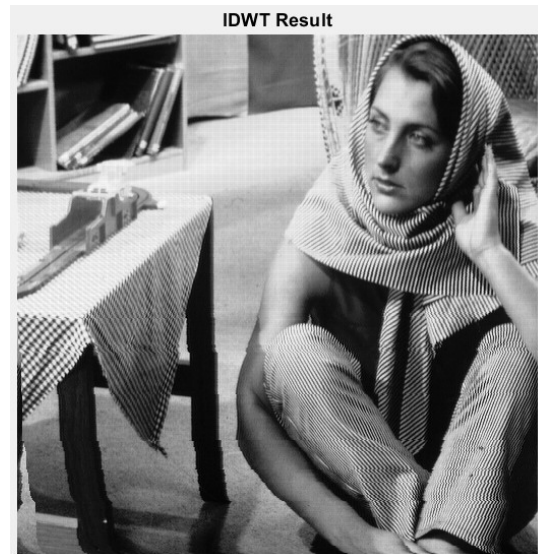
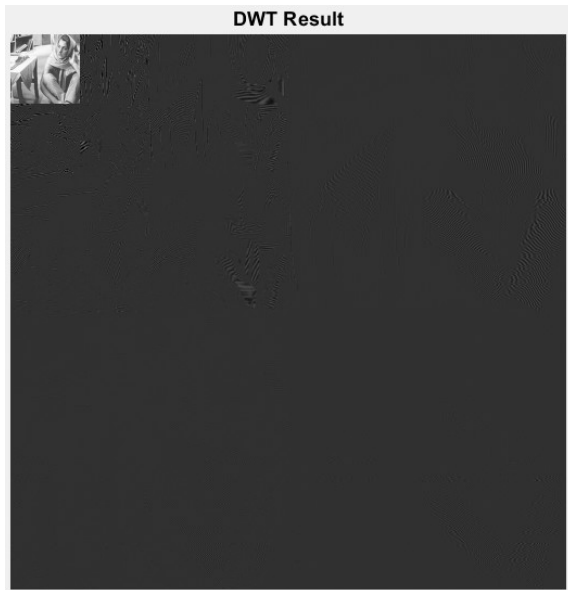
(2) Coefficient difference figure:



By above simulation, choosing word length equals to 9 will be a minimum best choice with fractional length 7 to meet PSNR value greater than 40dB for the following problem.

(b) Image processing:

Result:



Matlab code:

```
clc; clear; close all;
```

```
% Read image
```

```
OGimg = imread('woman.bmp'); % 512 X 512 image
```

```
DWT_out = zeros(512,512);
```

```
L = 512;
```

```
% dynamic range extension to prevent each level of calculating truncation
```

```
layer1_fixed = numerictype(true, 10, 0);
```

```
layer2_fixed = numerictype(true, 11, 0);
```

```
layer3_fixed = numerictype(true, 12, 0);
```

```
%% fixed point DWT in four parts of image: LL: left up HL: right up LH: left
```

```
%% down HH: right down. L: low freq. H: high freq.
```

```
% first-level 2D-DWT (row by column)
```

```
[out_l , out_h] = DWT_row_processing(L , OGimg , layer1_fixed);
```

```
[out_ll , out_hl , out_lh , out_hh] = DWT_column_processing(L , out_l , out_h ,  
layer1_fixed);
```

```
DWT_out(1 : 256 , 257 : 512) = out_hl;
```

```
DWT_out(257 : 512 , 1 : 256) = out_lh;
```

```
DWT_out(257 : 512 , 257 : 512) = out_hh;
```

```
% second-level 2D-DWT (row -> column)
```

```
[out_l_2 , out_h_2] = DWT_row_processing(256 , out_ll , layer2_fixed);
```

```
[out_ll_2 , out_hl_2 , out_lh_2 , out_hh_2] = DWT_column_processing(256 , out_l_2 ,  
out_h_2 , layer2_fixed);
```

```
DWT_out(1 : 128 , 129 : 256) = out_hl_2;
```

```
DWT_out(129 : 256 , 1 : 128) = out_lh_2;
```

```

DWT_out(129 : 256 , 129 : 256) = out_hh_2;

% third-level 2D-DWT (row -> column)
[out_l_3 , out_h_3] = DWT_row_processing(128 , out_ll_2 , layer3_fixed);
[out_ll_3 , out_hl_3 , out_lh_3 , out_hh_3] = DWT_column_processing(128 , out_l_3 ,
out_h_3 , layer3_fixed);
DWT_out(1 : 64 , 1 : 64) = out_ll_3;
DWT_out(1 : 64 , 65 : 128) = out_hl_3;
DWT_out(65 : 128 , 1 : 64) = out_lh_3;
DWT_out(65 : 128 , 65 : 128) = out_hh_3;

%% floating point IDWT in four parts of image
% first-level 2D-IDWT (column -> row)
[l_out_l_2 , l_out_h_2] = IDWT_column_processing(128 , out_ll_3 , out_lh_3 , out_hl_3 ,
out_hh_3);
[IDWT_img_2] = IDWT_row_processing(128 , l_out_l_2 , l_out_h_2);

% second-level 2D-IDWT (column -> row)
[l_out_l_1 , l_out_h_1] =
IDWT_column_processing(256 , IDWT_img_2 , out_lh_2 , out_hl_2 , out_hh_2);
[IDWT_img_1] = IDWT_row_processing(256 , l_out_l_1 , l_out_h_1);

% third-level 2D-IDWT (column -> row)
[l_out_l , l_out_h] = IDWT_column_processing(L , IDWT_img_1 , out_lh , out_hl , out_hh);
[IDWT_img] = IDWT_row_processing(L , l_out_l , l_out_h);

%% PSNR calculation, MAXI = 255 for 8-bit image
MSE = 0;
for i = 1 : 512
    for j = 1 : 512
        MSE = MSE + ((double(OGimg(i,j)) - double(IDWT_img(i,j))) ^ 2);
    end
end
MSE = MSE / (512 ^ 2);
PSNR = 10 * (log10((255 ^ 2) / MSE));
fprintf('PSNR = %3.4f dB\n', PSNR);

WL = 9; % fixed point word length = 9
FL = 7; % fixed point fractional length = 7

%% Bit True Algorithm sheet
fprintf('\n-----\n');
fprintf('    Final 3-Level 2-D DWT Bit-True Design Summary\n');
fprintf('-----\n');
fprintf('%-25s | %-10s | %-10s\n', 'Variable Name', 'Word Length', 'Fractional');
fprintf('-----\n');
fprintf('%-25s | %-11d | %-10d\n', 'Filter Coefficients (h,g)', 9, 7);
fprintf('%-25s | %-11d | %-10d\n', 'Level 1 DWT (out_l/h)', WL, FL);
fprintf('%-25s | %-11d | %-10d\n', 'Level 2 DWT (out_l_2/h_2)', WL+1, FL);
fprintf('%-25s | %-11d | %-10d\n', 'Level 3 DWT (out_l_3/h_3)', WL+2, FL);
fprintf('-----\n');

```

```

%% Image Processing Result
figure(1)
imshow(mat2gray(double(DWT_out)));
title('DWT Result');

figure(2)
imshow(mat2gray(double(IDWT_img)));
title('IDWT Result');

%% DWT row processing function (down-sampling)
function [out_l ,out_h] = DWT_row_processing(L , img , fixed_size)

% Analysis filter coefficients
h_floating = [ 0.037828455507; -0.023849465020; -0.110624404418; 0.377402855613; 0.852698679009;
0.377402855613; -0.110624404418; -0.023849465020; 0.037828455507];
g_floating = [-0.064538882629; 0.040689417609; 0.418092273222; -0.788485616406; 0.418092273222;
0.040689417609; -0.064538882629];

% Fixed point coefficients (find minimum word length)
h = fi(h_floating, 1, 9, 7); % fixed point word length = 9, fractional length = 7
g = fi(g_floating, 1, 9, 7); % fixed point word length = 9, fractional length = 7

% Symmetric extension for image boundary
DWT_row_l = zeros(L, L + 8, 'like', fi([], fixed_size)); % 左右各補 4 個點，'like'為按照 fixed
型別創造 0 矩陣
DWT_row_l = [img( : , 5) img( : , 4) img( : , 3) img( : , 2) img img( : , L - 1) img( : , L - 2)
img( : , L - 3) img( : , L - 4)];
DWT_row_h = zeros(L, L + 6, 'like', fi([], fixed_size)); % 左右各補 3 個點
DWT_row_h = [img( : , 4) img( : , 3) img( : , 2) img img( : , L - 1) img( : , L - 2) img( : , L -
3)];

% Output image
for i = 1 : L
    % lowpass filter for odd output data
    temp_row_l = conv(DWT_row_l(i , : ) , h); % 1D convolution for each row patch and filter
coefficient
    % from the 9th patch, stride = 2, to the (L+7)th patch for full conv
    out_l(i , 1 : (L / 2)) = fi(temp_row_l(1 , 9 : 2 : (L + 7)) ,fixed_size);

    % highpass filter for even output data
    temp_row_h = conv(DWT_row_h(i , : ) , g);
    % from the 8th patch, stride = 2, to the (L+6)th patch for full conv
    out_h(i , 1 : (L / 2)) = fi(temp_row_h(1 , 8 : 2 : (L+6)) , fixed_size);
end
end

%% DWT column processing function (down-sampling)
function [out_ll , out_hl , out_lh , out_hh] =
DWT_column_processing(L ,input_l ,input_h ,fixed_size)

% Analysis filter coefficients
h_floating = [ 0.037828455507; -0.023849465020; -0.110624404418; 0.377402855613; 0.852698679009;
0.377402855613; -0.110624404418; -0.023849465020; 0.037828455507];

```

```
g_floating = [-0.064538882629; 0.040689417609; 0.418092273222; -0.788485616406; 0.418092273222;
0.040689417609; -0.064538882629];
```

```
% Fixed point coefficients (find minimum word length)
h = fi(h_floating, 1, 9, 7); % fixed point word length = 9, fractional length = 7
g = fi(g_floating, 1, 9, 7); % fixed point word length = 9, fractional length = 7
```

```
% Symmetric extension for image boundary, LL HL 上下各補 4 個點(9-tap low pass) 、 LH
HH 上下各補 3 個點(7-tap high pass)
DWT_col_ll = zeros(L + 8, L / 2, 'like', fi([], fixed_size));
DWT_col_ll = [input_l(5, :); input_l(4, :); input_l(3, :); input_l(2, :); input_l; input_l(L -
1, :); input_l(L - 2, :); input_l(L - 3, :); input_l(L - 4, :)];
```

```
DWT_col_lh = zeros(L + 6, L / 2, 'like', fi([], fixed_size));
DWT_col_lh = [input_l(4, :); input_l(3, :); input_l(2, :); input_l; input_l(L - 1, :); input_l(L
- 2, :); input_l(L - 3, :)];
```

```
DWT_col_hl = zeros(L + 8, L / 2, 'like', fi([], fixed_size));
DWT_col_hl = [input_h(5, :); input_h(4, :); input_h(3, :); input_h(2, :); input_h;
input_h(L - 1, :); input_h(L - 2, :); input_h(L - 3, :); input_h(L - 4, :)];
```

```
DWT_col_hh = zeros(L + 6, L / 2, 'like', fi([], fixed_size));
DWT_col_hh = [input_h(4, :); input_h(3, :); input_h(2, :); input_h; input_h(L - 1, :);
input_h(L - 2, :); input_h(L - 3, :)];
```

```
% Output image
```

```
for i = 1 : L/2
```

```
    % lowpass filter for odd output data
```

```
    temp_col_ll = conv(DWT_col_ll(:, i), h);
```

```
    out_ll(1 : (L / 2), i) = fi(temp_col_ll(9 : 2 : (L + 7), 1), fixed_size);
```

```
    temp_col_hl = conv(DWT_col_hl(:, i), h);
```

```
    out_hl(1 : (L / 2), i) = fi(temp_col_hl(9 : 2 : (L + 7), 1), fixed_size);
```

```
    % highpass filter for even output data
```

```
    temp_col_lh = conv(DWT_col_lh(:, i), g);
```

```
    out_lh(1 : (L / 2), i) = fi(temp_col_lh(8 : 2 : (L + 6), 1), fixed_size);
```

```
    temp_col_hh = conv(DWT_col_hh(:, i), g);
```

```
    out_hh(1 : (L / 2), i) = fi(temp_col_hh(8 : 2 : (L + 6), 1), fixed_size);
```

```
end
```

```
end
```

```
%% IDWT column processing function (up-sampling)
```

```
function [out_l, out_h] = IDWT_column_processing(L, input_ll, input_lh, input_hl,
input_hh)
```

```
% Synthesis filter coefficients
```

```
q = [-0.064538882629; -0.040689417609; 0.418092273222; 0.788485616406; 0.418092273222; -
0.040689417609; -0.064538882629];
```

```
p = [-0.037828455507; -0.023849465020; 0.110624404418; 0.377402855613; -0.852698679009;
0.377402855613; 0.110624404418; -0.023849465020; -0.037828455507];
```

```

% Symmetric extension for image boundary, LL HL 上下各 3 個點(7-tap low pass)、LH HH
上下各 4 個點(9-tap high pass)
IDWT_col_ll = zeros(L + 6, L / 2); % 先建立 0 矩陣，後續填值可自動補 0，不須額外插值
IDWT_col_ll(4 : 2 : (L + 2), :) = input_ll; % Insert input from the 4th patch, stride = 2, to
the (L+2)th patch
IDWT_col_ll(1 : 3, :) = [IDWT_col_ll(7, :); IDWT_col_ll(6, :); IDWT_col_ll(5, :)]; % 上部的
對稱延展對應
IDWT_col_ll((L + 4) : (L + 6), :) = [IDWT_col_ll(L + 2, :); IDWT_col_ll(L + 1, :);
IDWT_col_ll(L + 0, :)]; % 下部的對稱延展對應

IDWT_col_hl = zeros(L + 6, L / 2);
IDWT_col_hl(4 : 2 : (L + 2), :) = input_hl;
IDWT_col_hl(1 : 3, :) = [IDWT_col_hl(7, :); IDWT_col_hl(6, :); IDWT_col_hl(5, :)];
IDWT_col_hl((L + 4) : (L + 6), :) = [IDWT_col_hl(L + 2, :); IDWT_col_hl(L + 1, :);
IDWT_col_hl(L + 0, :)];

IDWT_col_lh = zeros(L + 8, L / 2);
IDWT_col_lh(6 : 2 : (L + 4), :) = input_lh;
IDWT_col_lh(1 : 4, :) = [IDWT_col_lh(9, :); IDWT_col_lh(8, :); IDWT_col_lh(7, :);
IDWT_col_lh(6, :)];
IDWT_col_lh((L + 5) : (L + 8), :) = [IDWT_col_lh(L + 3, :); IDWT_col_lh(L + 2, :);
IDWT_col_lh(L + 1, :); IDWT_col_lh(L + 0, :)];

IDWT_col_hh = zeros(L + 8, L / 2);
IDWT_col_hh(6 : 2 : (L + 4), :) = input_hh;
IDWT_col_hh(1 : 4, :) = [IDWT_col_hh(9, :); IDWT_col_hh(8, :); IDWT_col_hh(7, :);
IDWT_col_hh(6, :)];
IDWT_col_hh((L + 5) : (L + 8), :) = [IDWT_col_hh(L + 3, :); IDWT_col_hh(L + 2, :);
IDWT_col_hh(L + 1, :); IDWT_col_hh(L + 0, :)];

% Output image
for i = 1 : (L / 2)
    % lowpass filter for odd output data
    temp_col_ll = conv(IDWT_col_ll(:, i), q);
    out_ll(1 : L, i) = temp_col_ll(7 : L + 6, 1);

    temp_col_hl = conv(IDWT_col_hl(:, i), q);
    out_hl(1 : L, i) = temp_col_hl(7 : L + 6, 1);

    % highpass filter for even output data
    temp_col_lh = conv(IDWT_col_lh(:, i), p);
    out_lh(1 : L, i) = temp_col_lh(9 : L + 8, 1);

    temp_col_hh = conv(IDWT_col_hh(:, i), p);
    out_hh(1 : L, i) = temp_col_hh(9 : L + 8, 1);
end
out_l = out_ll + out_lh;
out_h = out_hl + out_hh;
end

%% IDWT row processing function (up-sampling)
function [IDWT_result] = IDWT_row_processing(L, input_l, input_h)

```

% Synthesis filter coefficients

```
% Symmetric extension for image boundary
```

```
IDWT_row_h = zeros(L ,L + 8);
IDWT_row_h( : ,6 : 2 : (L + 4)) = input_h;
IDWT_row_h( : ,1 : 4) = [IDWT_row_h( : ,9) IDWT_row_h( : ,8) IDWT_row_h( : ,7)
IDWT_row_h( : ,6)];
IDWT_row_h( : ,(L + 5) : (L + 8)) = [IDWT_row_h( : , L + 3 ) IDWT_row_h( : , L + 2 )
IDWT_row_h( : , L + 1 ) IDWT_row_h( : , L + 0 )];
```

```
for i = 1 : L
```

```
% highpass filter for even output data
temp_h = conv(IDWT_row_h(i , : ) ,p);
out_h(i ,1 : L) = temp_h(1 ,9 : L + 8);
```

end

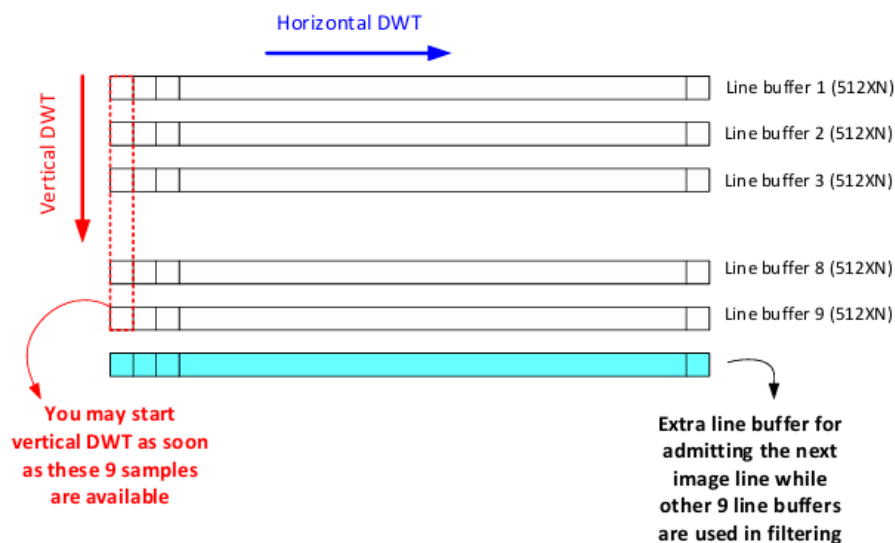


Fig. 2 Efficient computing scheme and transpose memory design

Q2. Develop a hardware design of 2-D DWT bit true algorithm you derive in Q1. Employ the architecture shown in Fig. 1.

- **Note 3:** You don't need to calculate every input sample and then discard every other output, i.e. down sample the results. Instead, you should calculate the

useful outputs only.

- You need a temporary memory storage (transpose memory) to keep the result of horizontal DWTs before the start of vertical DWTs. Please design your memory configuration, i.e. how many memory modules and the depth and width of each memory module.
- **Note 4:** If you perform vertical DWTs only **after** all horizontal DWTs are completed, it means you need to buffer almost the entire 512X512 image in your on-chip memory modules. The area overhead would be too high. Refer to Fig. 2, a better way of designing your transpose memory would be to buffer just a few lines of horizontal DWT results and start your vertical DWT right after you have sufficient input samples, i.e., 9 samples for low pass filtering.
- After 1st level of decomposition, you can write HL, LH and HH sub-bands data to external memory to reduce the on-chip memory size. For simplicity, you don't have to implement this part in your design.
- Please code your design in Verilog and conduct simulation to verify your design. Use the fixed-point simulation results obtained in Q1 as the golden pattern of testbench. Your Verilog simulation results should be identical to fixed point simulation results.
- Synthesize your design and report the following items. You may choose either TSMC ADFP virtual process or AMD (Xilinx) FPGA XC7Z020 as your target technology.
 - 1) For cell based design, report equivalent gate count or area, please also indicate the percentages of combinational circuit (filter) and memory modules
 - 2) For FPGA design, give resource utilization table showing the numbers of FFs, LUTs, DSP slices, and block RAMs used in your design.
 - 3) Number of clock cycles to complete the entire 2-D DWT computations
 - 4) Maximum working frequency

(a) Implement the fixed-point DWT in verilog code:

Using FPGA xc7z020clg400-1 to synthesize. Following are the synthesized result:

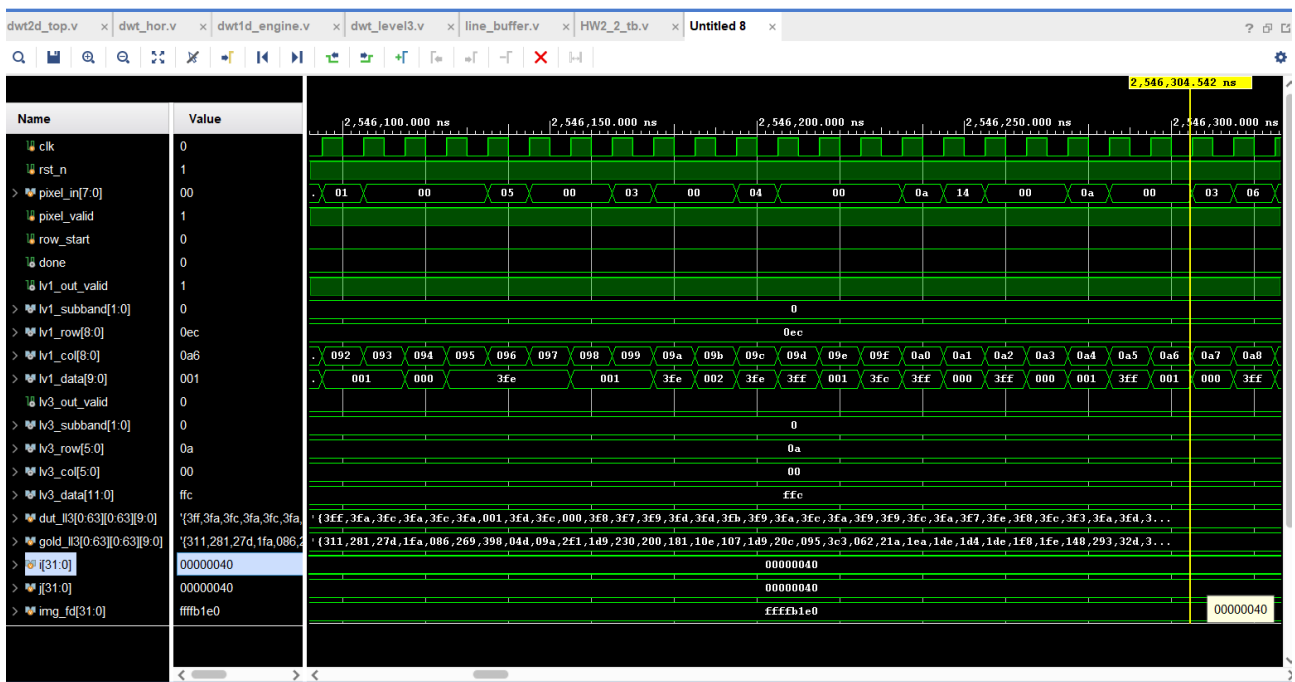
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology
✓ synth_1	constrs_1	synth_design Complete!									
✓ impl_1	constrs_1	route_design Complete!	1.740	0.000	0.055	0.000		0.000	0.125	0	291 Warn

LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy	Report Strategy	Part
4948	717	9.5	0	21	4/13/26, 2:21 PM	00:01:18	Vivado Synthesis Defaults (Vivado Synthesis 2024)	Vivado Synthesis Default Reports (Vivado Synthesis 2024)	xc7z020clg400-1
4794	736	9.5	0	21	4/13/26, 2:27 PM	00:01:38	Vivado Implementation Defaults (Vivado Implementation 2024)	Vivado Implementation Default Reports (Vivado Implementation 2024)	xc7z020clg400-1

(b) **timing constraint:** create_clock -period **40.000** -name clk -waveform {0.000 5.000}
[get_ports clk]

(c) Maximum working frequency: $1 / (40 - 1.74) = 0.0261\text{MHz}$

(d) waveform:



Verilog code:

```

1 // =====
2 // dwtd_engine.v , parameters in module synthesize easily
3 // =====
4 module dwtd_engine #(
5     parameter OUT_W = 10, // signed output width (Level 1: 10, Level 2: 11, Level 3: 12)
6     parameter WL = 9, // signed input sample width from
7     parameter FL = 7 // Fractional Length from design of Q1
8 ) (
9     input wire clk,
10    input wire rst_n,
11    input wire [WL*9-1:0] win_l_in, // 9-sample sliding window for low pass filter (h)
12    input wire [WL*7-1:0] win_h_in, // 7-sample sliding window for high pass filter (g)
13    input wire valid_in,
14    output reg signed [OUT_W-1:0] out_l,
15    output reg signed [OUT_W-1:0] out_h,
16    output reg valid_out
17 );
18
19 // fixed-point Analysis filter coefficient LUT (multiply by 2^7=128)
20 // h_floating = [ 0.037828455507; -0.023849465020; -0.110624404418; 0.377402855613; 0.852698679009; 0
21
22 localparam signed [8:0] H0 = 9'sd5;
23 localparam signed [8:0] H1 = -9'sd3;
24 localparam signed [8:0] H2 = -9'sd14;
25 localparam signed [8:0] H3 = 9'sd48;
26 localparam signed [8:0] H4 = 9'sd109; // middle term
27 localparam signed [8:0] H5 = 9'sd48;
28 localparam signed [8:0] H6 = -9'sd14;
29 localparam signed [8:0] H7 = -9'sd3;
30 localparam signed [8:0] H8 = 9'sd5;
31

```

```

33 localparam signed [8:0] G0 = -9'sd8;
34 localparam signed [8:0] G1 = 9'sd5;
35 localparam signed [8:0] G2 = 9'sd54;
36 localparam signed [8:0] G3 = -9'sd101; // middle term
37 localparam signed [8:0] G4 = 9'sd54;
38 localparam signed [8:0] G5 = 9'sd5;
39 localparam signed [8:0] G6 = -9'sd8;
40
41 wire signed [WL-1:0] d_l [0:8]; // low pass filter conv. term
42 wire signed [WL-1:0] d_h [0:6]; // high pass filter conv. term
43
44 genvar i;
45 generate
46     for (i = 0; i < 9; i = i + 1) begin assign d_l[i] = win_l_in[i*WL +: WL]; end
47     for (i = 0; i < 7; i = i + 1) begin assign d_h[i] = win_h_in[i*WL +: WL]; end
48 endgenerate
49
50 // reg signed [23:0] sum_l, sum_h;
51
52 always @(posedge clk or negedge rst_n) begin
53     if (!rst_n) begin
54         out_l <= 0; out_h <= 0; valid_out <= 0;
55     end else if (valid_in) begin
56         // Low-pass Convolution
57         out_l <= (d_l[0]*H0 + d_l[1]*H1 + d_l[2]*H2 + d_l[3]*H3 + d_l[4]*H4 +
58                 d_l[5]*H5 + d_l[6]*H6 + d_l[7]*H7 + d_l[8]*H8) >>> FL;
59         // High-pass Convolution
60         out_h <= (d_h[0]*G0 + d_h[1]*G1 + d_h[2]*G2 + d_h[3]*G3 + d_h[4]*G4 +
61                 d_h[5]*G5 + d_h[6]*G6) >>> FL;
62         valid_out <= 1'b1;
63     end else begin
64         valid_out <= 1'b0;
65     end
66 end
67
68 endmodule

```

```

1 module line_buffer #(
2     parameter WIDTH = 10, // data bits
3     parameter DEPTH = 512 // memory depth
4 ) (
5     input wire clk,
6     input wire wr_en, // write enable
7     input wire [$clog2(DEPTH)-1:0] wr_addr, // write address
8     input wire [WIDTH-1:0] wr_data, // write data
9     input wire [$clog2(DEPTH)-1:0] rd_addr, // read address
10    output reg [WIDTH-1:0] rd_data // read data
11 );
12
13 reg [WIDTH-1:0] mem [0:DEPTH-1]; // Memory declaration
14
15 always @(posedge clk) begin
16     if (wr_en) mem[wr_addr] <= wr_data; // write data into memory
17     rd_data <= mem[rd_addr]; // read data from memory
18 end
19
20 endmodule

```

```

2 // dwt_hor.v, first processing horizontal DWT processing
3 // =====
4 module horiz_dwt #(
5     parameter L      = 512,
6     parameter IN_W    = 9,    // input pixel bit width
7     parameter OUT_W   = 10    // output pixel bit width
8 ) (
9     input                clk,
10    input                rst_n,
11    input    signed [IN_W-1:0] pixel_in,    // input pixel
12    input                pixel_valid,    // input valid
13    input                row_start,    // change to the new row
14    output wire signed [OUT_W-1:0] out_l,    // low pass output coef.
15    output wire signed [OUT_W-1:0] out_h,    // high pass output coef.
16    output wire                out_valid    // output valid
17 );
18
19 localparam LOG2L = $clog2(L + 10); // 522 pixels needs 10-bits above
20
21 // FSM definition
22 localparam ST_IDLE      = 3'd0; // 閒置狀態
23 localparam ST_LEFT_PAD  = 3'd1; // left boundray processing
24 localparam ST_PROCESS   = 3'd2; // middle image processing
25 localparam ST_RIGHT_PAD = 3'd3; // right boundary processing
26 localparam ST_FINISH    = 3'd4; // DONE
27
28 reg [2:0] current_state, next_state; // FSM state transiton
29 reg [15:0] in_cnt; // input pixel counter
30 reg [3:0] pad_cnt; // extension pixel counter
31 reg [15:0] ext_cnt; // pixel in filter amount
32 reg                out_phase; // each two down-sampling phase
33
34 reg signed [IN_W-1:0] sr9 [0:8]; // 9-tap sliding window
35 reg signed [IN_W-1:0] left_buf [0:4]; // first 5 pixels for left boundary
36 reg signed [IN_W-1:0] right_buf [0:4]; // last 5 pixels for right boundary
37
38 // Symmetric extension signal
39 reg signed [IN_W-1:0] ext_sample; // samples into window
40 reg                ext_valid; // extension valid signal
41
42 integer i;
43
44 // FSM current state logic
45 always @(posedge clk or negedge rst_n) begin
46     if (!rst_n) current_state <= ST_IDLE;
47     else        current_state <= next_state;
48 end
49
50 // FSM next state logic
51 always @(*) begin
52     case (current_state)
53         ST_IDLE:      if (row_start) next_state = ST_LEFT_PAD; // row_start 開始偵測
54         ST_LEFT_PAD:  if (in_cnt == 5) next_state = ST_PROCESS; // 存好5個前面pixel後開始處理
55         ST_PROCESS:   if (in_cnt == L) next_state = ST_RIGHT_PAD; // 處理完原始pixel後開始處理右邊界
56         ST_RIGHT_PAD: if (pad_cnt == 4) next_state = ST_FINISH; // 右邊邊界補齊後進入結束狀態
57         ST_FINISH:    next_state = ST_IDLE;
58         default:      next_state = ST_IDLE;
59     endcase
60 end

```

```

62 // input pixel counter and boundary buffer processing
63 always @(posedge clk or negedge rst_n) begin
64     if (!rst_n) begin
65         in_cnt <= 0;
66         for (i = 0; i < 5; i = i + 1) begin
67             left_buf[i] <= 0;
68             right_buf[i] <= 0;
69         end
70     end else begin
71         case (current_state)
72             ST_IDLE: in_cnt <= 0;
73             ST_LEFT_PAD, ST_PROCESS: begin
74                 if (pixel_valid && in_cnt < L) begin
75                     in_cnt <= in_cnt + 1;
76                     if (in_cnt < 5) left_buf[in_cnt] <= pixel_in;
77                     right_buf[0] <= pixel_in;
78                     for (i = 1; i < 5; i = i + 1) right_buf[i] <= right_buf[i-1];
79                 end
80             end
81             ST_RIGHT_PAD: in_cnt <= 0;
82         endcase
83     end
84 end

```

```

86 // extension sample stream
87 always @(posedge clk or negedge rst_n) begin
88     if (!rst_n) begin
89         ext_sample <= 0; ext_valid <= 0; pad_cnt <= 0;
90     end else begin
91         ext_valid <= 0;
92         case (current_state)
93             ST_LEFT_PAD: begin
94                 if (in_cnt == 5) begin
95                     ext_sample <= left_buf[4-pad_cnt];
96                     ext_valid <= 1;
97                     pad_cnt <= pad_cnt + 1;
98                     if (pad_cnt == 4) pad_cnt <= 0;
99                 end
100             end
101             ST_PROCESS: begin
102                 if (pixel_valid) begin
103                     ext_sample <= pixel_in;
104                     ext_valid <= 1;
105                 end
106             end
107             ST_RIGHT_PAD: begin
108                 if (pad_cnt < 4) begin
109                     ext_sample <= right_buf[pad_cnt+1];
110                     ext_valid <= 1;
111                     pad_cnt <= pad_cnt + 1;
112                 end
113             end
114             default: pad_cnt <= 0;
115         endcase
116     end
117 end

```

```

119 // sliding window
120 always @(posedge clk or negedge rst_n) begin
121     if (!rst_n) begin
122         for (i=0; i<9; i=i+1) sr9[i] <= 0;
123         ext_cnt <= 0;
124         out_phase <= 0;
125     end else if (row_start) begin
126         ext_cnt <= 0;
127         out_phase <= 0;
128         for (i=0; i<9; i=i+1) sr9[i] <= 0;
129     end else if (ext_valid) begin
130         for (i=8; i>0; i=i-1) sr9[i] <= sr9[i-1];
131         sr9[0] <= ext_sample;
132         if (ext_cnt < 65535) ext_cnt <= ext_cnt + 1;
133         if (ext_cnt >= 8) out_phase <= ~out_phase;
134     end
135 end
136
137 // Bus
138 wire [IN_W*9-1:0] win_l_flat;
139 wire [IN_W*7-1:0] win_h_flat;
140
141 genvar g;
142 generate
143     for (g=0; g<9; g=g+1) begin assign win_l_flat[g*IN_W +: IN_W] = sr9[g]; end
144     for (g=0; g<7; g=g+1) begin assign win_h_flat[g*IN_W +: IN_W] = sr9[g+1]; end
145 endgenerate

147 dwt1d_engine #(.OUT_W(OUT_W), .WL(IN_W)) u_engine (
148     .clk(clk),
149     .rst_n(rst_n),
150     .win_l_in(win_l_flat),
151     .win_h_in(win_h_flat),
152     .valid_in(ext_valid && (ext_cnt >= 8) && (out_phase == 1)),
153     .out_l(out_l),
154     .out_h(out_h),
155     .valid_out(out_valid)
156 );
157
158 endmodule

```

```

1 module dwt_level3 (
2     input wire      clk,
3     input wire      rst_n,
4     input wire signed [9:0] data_in,    // LL1
5     input wire      valid_in,
6     output wire signed [11:0] data_out,  // LL3
7     output wire      valid_out,
8     output wire [1:0] subband_out,
9     output wire [5:0] row_out,
10    output wire [5:0] col_out
11 );
12
13 // Level 2: 256x256 LL1 -> 128x128 LL2
14 // 水平: horiz_dwt L=256, IN_W=10, OUT_W=11
15 // 垂直: 9條 line buffer + dwt1d_engine
16
17 // --- Level 2 Horizontal ---
18 wire signed [10:0] h2_l, h2_h;
19 wire      h2_v;
20 reg [7:0] c2_cnt; // 每行256個輸入像素計數
21
22 always @(posedge clk or negedge rst_n)
23     if (!rst_n) c2_cnt <= 0;
24     else if (valid_in) c2_cnt <= (c2_cnt == 255) ? 8'd0 : c2_cnt + 1;
25
26 horiz_dwt #(.L(256), .IN_W(10), .OUT_W(11)) u_h2 (
27     .clk(clk), .rst_n(rst_n),
28     .pixel_in(data_in), .pixel_valid(valid_in),
29     .row_start(valid_in && (c2_cnt == 0)),
30     .out_l(h2_l), .out_h(h2_h), .out_valid(h2_v)
31 );

```

```

33 // --- Level 2 Vertical Line Buffers (9條, 深度128, 寬度11) ---
34 // 每條儲存水平低通輸出的一行 (128個樣本)
35 reg [6:0] v2_wr_addr;
36 reg [3:0] v2_wr_line; // 0~8 循環
37 reg [8:0] v2_row_total; // 已寫入幾行
38
39 wire signed [10:0] v2_lb_l [0:8];
40 reg [6:0] v2_rd_addr;
41
42 genvar gv2;
43 generate
44     for (gv2 = 0; gv2 < 9; gv2 = gv2 + 1) begin : V2LB
45         line_buffer #(.WIDTH(11), .DEPTH(128)) lb_v2 (
46             .clk(clk),
47             .wr_en (h2_v && (v2_wr_line == gv2)),
48             .wr_addr(v2_wr_addr),
49             .wr_data(h2_l), // 只對 LL path 做垂直濾波
50             .rd_addr(v2_rd_addr),
51             .rd_data(v2_lb_l[gv2])
52         );
53     end
54 endgenerate
55
56 always @(posedge clk or negedge rst_n) begin
57     if (!rst_n) begin
58         v2_wr_addr <= 0; v2_wr_line <= 0; v2_row_total <= 0;
59     end else if (h2_v) begin
60         v2_wr_addr <= v2_wr_addr + 1;
61         if (v2_wr_addr == 127) begin
62             v2_wr_addr <= 0;
63             v2_row_total <= v2_row_total + 1;
64             v2_wr_line <= (v2_wr_line == 9) ? 4'd0 : v2_wr_line + 1;

```

```

96     for (gv2f = 0; gv2f < 7; gv2f = gv2f + 1) begin
97         assign v2_win_h_bus[gv2f*11 +: 11] = v2_lb_l[v2_lb_sel[gv2f+1]]; end
98     endgenerate
99
100     wire signed [10:0] v2_res_ll; // LL2 output
101     wire                v2_out_v;
102     dwtd_engine #(.WL(11), .OUT_W(11), .FL(7)) u_v2_vert (
103         .clk(clk), .rst_n(rst_n),
104         .win_l_in(v2_win_l_bus), .win_h_in(v2_win_h_bus),
105         .valid_in(v2_eng_valid),
106         .out_l(v2_res_ll), .out_h(/* HL2 unused */),
107         .valid_out(v2_out_v)
108     );
109
110     always @(posedge clk or negedge rst_n) begin
111         if (!rst_n) begin
112             v2_vstate <= VS_IDLE; v2_vrow <= 0; v2_vcol <= 0;
113             v2_eng_valid <= 0; v2_rd_addr <= 0;
114         end else begin
115             case (v2_vstate)
116             VS_IDLE: if (v2_row_total >= 5) begin
117                 v2_rd_addr <= 0; v2_vstate <= VS_PREP;
118             end
119             VS_PREP: begin
120                 v2_rd_addr <= 1; v2_eng_valid <= 1; v2_vstate <= VS_COMP;
121             end
122             VS_COMP: begin
123                 if (v2_vcol == 127) begin
124                     v2_eng_valid <= 0; v2_vcol <= 0; v2_rd_addr <= 0;
125                     if (v2_vrow == 127) begin
126                         v2_vstate <= VS_IDLE;
127                     end else begin

```



```

128             v2_vrow <= v2_vrow + 1; v2_vstate <= VS_WAIT;
129         end
130     end else begin
131         v2_vcol <= v2_vcol + 1;
132         v2_rd_addr <= v2_vcol + 2;
133     end
134 end
135 VS_WAIT: if (v2_row_total >= (v2_vrow << 1) + 5)
136     v2_vstate <= VS_PREP;
137 endcase
138 end
139 end
140
141 // Level 3: 128x128 LL2 -> 64x64 LL3
142 // 水平: horiz_dwt L=128, IN_W=11, OUT_W=12
143 // 垂直: 9條 line buffer + dwt1d_engine
144
145 // --- Level 3 Horizontal ---
146 wire signed [11:0] h3_l, h3_h;
147 wire             h3_v;
148 reg [6:0]        c3_cnt;
149
150 always @(posedge clk or negedge rst_n)
151     if (!rst_n) c3_cnt <= 0;
152     else if (v2_out_v) c3_cnt <= (c3_cnt == 127) ? 7'd0 : c3_cnt + 1;
153
154 horiz_dwt #(.L(128), .IN_W(11), .OUT_W(12)) u_h3 (
155     .clk(clk), .rst_n(rst_n),
156     .pixel_in(v2_res_l1), .pixel_valid(v2_out_v),
157     .row_start(v2_out_v && (c3_cnt == 0)),
158     .out_l(h3_l), .out_h(h3_h), .out_valid(h3_v)
159 );
160
161 // --- Level 3 Vertical Line Buffers (9條, 深度64, 寬度12) ---
162 reg [5:0] v3_wr_addr;
163 reg [3:0] v3_wr_line;
164 reg [7:0] v3_row_total;
165
166 wire signed [11:0] v3_lb_l [0:8];
167 reg [5:0] v3_rd_addr;
168
169 genvar gv3;
170 generate
171     for (gv3 = 0; gv3 < 9; gv3 = gv3 + 1) begin : V3LB
172         line_buffer #(.WIDTH(12), .DEPTH(64)) lb_v3 (
173             .clk(clk),
174             .wr_en (h3_v && (v3_wr_line == gv3)),
175             .wr_addr(v3_wr_addr),
176             .wr_data(h3_l),
177             .rd_addr(v3_rd_addr),
178             .rd_data(v3_lb_l[gv3])
179         );
180     end
181 endgenerate
182
183 always @(posedge clk or negedge rst_n) begin
184     if (!rst_n) begin
185         v3_wr_addr <= 0; v3_wr_line <= 0; v3_row_total <= 0;
186     end else if (h3_v) begin
187         v3_wr_addr <= v3_wr_addr + 1;
188         if (v3_wr_addr == 63) begin
189             v3_wr_addr <= 0;
190             v3_row_total <= v3_row_total + 1;
191             v3_wr_line <= (v3_wr_line == 9) ? 4'd0 : v3_wr_line + 1;
192         end

```

```

193     end
194 end
195
196 // --- Level 3 Vertical Control FSM ---
197 reg [1:0] v3_vstate;
198 reg [6:0] v3_vrow;
199 reg [5:0] v3_vcol;
200 reg      v3_eng_valid;
201 reg [3:0] v3_lb_sel [0:8];
202
203 integer kk3;
204 reg signed [10:0] ty3;
205 always @(*) begin
206     for (kk3 = 0; kk3 < 9; kk3 = kk3 + 1) begin
207         ty3 = $signed({1'b0, v3_vrow}) * 2 + kk3 - 4;
208         if (ty3 < 0) ty3 = -ty3;
209         else if (ty3 > 127) ty3 = 254 - ty3;
210         v3_lb_sel[kk3] = ty3 % 9;
211     end
212 end
213
214 wire [107:0] v3_win_l_bus; // 9 * 12 bits
215 wire [83:0] v3_win_h_bus; // 7 * 12 bits
216 genvar gv3f;
217 generate
218     for (gv3f = 0; gv3f < 9; gv3f = gv3f + 1) begin
219         assign v3_win_l_bus[gv3f*12 +: 12] = v3_lb_l[v3_lb_sel[gv3f]]; end
220     for (gv3f = 0; gv3f < 7; gv3f = gv3f + 1) begin
221         assign v3_win_h_bus[gv3f*12 +: 12] = v3_lb_l[v3_lb_sel[gv3f+1]]; end
222 endgenerate

```

```

224 wire signed [11:0] v3_res_ll;
225 wire            v3_out_v;
226 dwt1d_engine #(.WL(12), .OUT_W(12), .FL(7)) u_v3_vert (
227     .clk(clk), .rst_n(rst_n),
228     .win_l_in(v3_win_l_bus), .win_h_in(v3_win_h_bus),
229     .valid_in(v3_eng_valid),
230     .out_l(v3_res_ll), .out_h(/* HL3 unused */),
231     .valid_out(v3_out_v)
232 );
233
234 always @(posedge clk or negedge rst_n) begin
235     if (!rst_n) begin
236         v3_vstate <= VS_IDLE; v3_vrow <= 0; v3_vcol <= 0;
237         v3_eng_valid <= 0; v3_rd_addr <= 0;
238     end else begin
239         case (v3_vstate)
240             VS_IDLE: if (v3_row_total >= 5) begin
241                 v3_rd_addr <= 0; v3_vstate <= VS_PREP;
242             end
243             VS_PREP: begin
244                 v3_rd_addr <= 1; v3_eng_valid <= 1; v3_vstate <= VS_COMP;
245             end
246             VS_COMP: begin
247                 if (v3_vcol == 63) begin
248                     v3_eng_valid <= 0; v3_vcol <= 0; v3_rd_addr <= 0;
249                     if (v3_vrow == 63) begin
250                         v3_vstate <= VS_IDLE;
251                     end else begin
252                         v3_vrow <= v3_vrow + 1; v3_vstate <= VS_WAIT;
253                     end
254                 end else begin
255                     v3_vcol <= v3_vcol + 1;

```

```

256 |                                     v3_rd_addr <= v3_vcol + 2;
257 |                                     end
258 |                                     end
259 |                                     VS_WAIT: if (v3_row_total >= (v3_vrow << 1) + 5)
260 |                                         v3_vstate <= VS_PREP;
261 |                                     endcase
262 |                                     end
263 |                                 end
264 |
265 | // --- Output ---
266 | reg [5:0] r3, c3;
267 | always @(posedge clk) begin
268 |     if (!rst_n) begin {r3, c3} <= 0; end
269 |     else if (v3_out_v) begin
270 |         c3 <= c3 + 1;
271 |         if (c3 == 63) r3 <= r3 + 1;
272 |     end
273 | end
274 |
275 | assign data_out    = v3_res_l1;
276 | assign valid_out   = v3_out_v;
277 | assign subband_out = 2'b00;
278 | assign row_out     = r3;
279 | assign col_out     = c3;
280 |
281 | endmodule

```

```

1 | module dwt2d_top #(
2 |     parameter IMG_SIZE = 512
3 | ) (
4 |     input  wire      clk,
5 |     input  wire      rst_n,
6 |
7 |     // Streaming input
8 |     input  wire [7:0] pixel_in,
9 |     input  wire      pixel_valid,
10 |    input  wire      row_start,
11 |
12 |    output reg        done,
13 |
14 |    // Level-1 sub-band output
15 |    output wire        lv1_out_valid,
16 |    output wire [1:0]  lv1_subband,
17 |    output wire [8:0]  lv1_row,
18 |    output wire [8:0]  lv1_col,
19 |    output wire signed [9:0] lv1_data,
20 |
21 |    // Level-3 sub-band output
22 |    output wire lv3_out_valid,
23 |    output wire [1:0] lv3_subband,
24 |    output wire [5:0] lv3_row,
25 |    output wire [5:0] lv3_col,
26 |    output wire signed [11:0] lv3_data
27 | );
28 |
29 | // level-1 Horizontal DWT
30 | wire signed [9:0] h1_out_l, h1_out_h;
31 | wire              h1_out_valid;

```

```

32 horiz_dwt #(.L(512), .IN_W(9), .OUT_W(10)) u_horiz1 (
33     .clk(clk), .rst_n(rst_n), .pixel_in({1'b0, pixel_in}),
34     .pixel_valid(pixel_valid), .row_start(row_start),
35     .out_l(h1_out_l), .out_h(h1_out_h), .out_valid(h1_out_valid)
36 );
37
38 // Line Buffers
39 localparam NLB = 10;
40 reg [7:0] hcol_cnt;
41 reg [3:0] lb_wr_line;
42 reg [9:0] hrow_total;
43 wire signed [9:0] lb_rd_data_l [0:9];
44 wire signed [9:0] lb_rd_data_h [0:9];
45 reg [7:0] lb_rd_addr;
46
47 genvar gi;
48 generate
49     for (gi=0; gi<NLB; gi=gi+1) begin : LB_INST
50         line_buffer #(.WIDTH(10), .DEPTH(256)) lb_l_inst (
51             .clk(clk), .wr_en(h1_out_valid && (lb_wr_line == gi)),
52             .wr_addr(hcol_cnt), .wr_data(h1_out_l), .rd_addr(lb_rd_addr), .rd_data(lb_rd_data_l[gi])
53         );
54         line_buffer #(.WIDTH(10), .DEPTH(256)) lb_h_inst (
55             .clk(clk), .wr_en(h1_out_valid && (lb_wr_line == gi)),
56             .wr_addr(hcol_cnt), .wr_data(h1_out_h), .rd_addr(lb_rd_addr), .rd_data(lb_rd_data_h[gi])
57         );
58     end
59 endgenerate
60
61 always @(posedge clk or negedge rst_n) begin
62     if (!rst_n) begin hcol_cnt<=0; lb_wr_line<=0; hrow_total<=0; end
63     else if (h1_out_valid) begin
64         hcol_cnt <= hcol_cnt + 1;
65         if (hcol_cnt == 255) begin
66             hcol_cnt <= 0; hrow_total <= hrow_total + 1;
67             lb_wr_line <= (lb_wr_line == 9) ? 4'd0 : lb_wr_line + 4'd1;
68         end
69     end
70 end
71
72 // Vertical Control & FSM Registers
73 reg [2:0] vstate;
74 localparam VS_IDLE = 3'd0;
75 localparam VS_PREPARE = 3'd1;
76 localparam VS_COMPUTE = 3'd2;
77 localparam VS_WAIT = 3'd3;
78 localparam VS_FINISH = 3'd4;
79
80 reg [8:0] vrow_idx;
81 reg [7:0] vcol_idx;
82 reg [1:0] subband_sel;
83 reg      v_eng_valid;
84
85 reg signed [10:0] ty;
86 reg [3:0] v_lb_sel [0:8];
87 integer k_idx;
88
89 always @(*) begin
90     for (k_idx = 0; k_idx < 9; k_idx = k_idx + 1) begin
91         ty = (vrow_idx << 1) + k_idx - 4; // vrow_idx * 2
92         if (ty < 0) ty = -ty;
93         else if (ty > 511) ty = 1022 - ty;
94         v_lb_sel[k_idx] = ty % 10;
95     end

```

```

96 end
97
98 // Flattening & Vertical Engines
99 wire [89:0] vwin_l_bus, vwin_hl_bus;
100 wire [69:0] vwin_h_bus, vwin_hh_bus;
101 generate
102     for (gi=0; gi<9; gi=gi+1) begin : FLAT_9
103         assign vwin_l_bus[gi*10 +: 10] = lb_rd_data_l[v_lb_sel[gi]];
104         assign vwin_hl_bus[gi*10 +: 10] = lb_rd_data_h[v_lb_sel[gi]];
105     end
106     for (gi=0; gi<7; gi=gi+1) begin : FLAT_7
107         assign vwin_h_bus[gi*10 +: 10] = lb_rd_data_l[v_lb_sel[gi+1]];
108         assign vwin_hh_bus[gi*10 +: 10] = lb_rd_data_h[v_lb_sel[gi+1]];
109     end
110 endgenerate
111
112 wire signed [9:0] res_ll, res_lh, res_hl, res_hh;
113 wire v_out_valid_l, v_out_valid_h;
114
115 dwt1d_engine #(.WL(10), .OUT_W(10)) u_vert_L (
116     .clk(clk), .rst_n(rst_n), .win_l_in(vwin_l_bus), .win_h_in(vwin_h_bus),
117     .valid_in(v_eng_valid), .out_l(res_ll), .out_h(res_lh), .valid_out(v_out_valid_l)
118 );
119
120 dwt1d_engine #(.WL(10), .OUT_W(10)) u_vert_H (
121     .clk(clk), .rst_n(rst_n), .win_l_in(vwin_hl_bus), .win_h_in(vwin_hh_bus),
122     .valid_in(v_eng_valid), .out_l(res_hl), .out_h(res_hh), .valid_out(v_out_valid_h)
123 );

```

```

125 // level-3 Instance (LL1 Path)
126 dwt_level3 u_l3 (
127     .clk(clk), .rst_n(rst_n),
128     .data_in(res_ll), .valid_in(v_out_valid_l),
129     .data_out(lv3_data), .valid_out(lv3_out_valid),
130     .subband_out(lv3_subband), .row_out(lv3_row), .col_out(lv3_col)
131 );
132
133 // FSM
134 always @(posedge clk or negedge rst_n) begin
135     if (!rst_n) begin
136         vstate    <= VS_IDLE;
137         vrow_idx   <= 0;
138         vcol_idx   <= 0;
139         done       <= 0;
140         lb_rd_addr <= 0;
141         v_eng_valid <= 0;
142         subband_sel <= 0;
143     end else begin
144         case (vstate)
145             VS_IDLE: begin
146                 if (hrow_total >= 5) begin
147                     lb_rd_addr <= 0;
148                     vstate    <= VS_PREPARE;
149                 end
150             end
151             VS_PREPARE: begin
152                 // 預留一拍讓 SRAM 讀出第一筆資料
153                 lb_rd_addr <= 1;
154                 v_eng_valid <= 1;
155                 vstate    <= VS_COMPUTE;
156             end

```

```

157 VS_COMPUTE: begin
158     if (vcol_idx == 255) begin
159         v_eng_valid <= 0;
160         vcol_idx    <= 0;
161         lb_rd_addr  <= 0;
162     if (vrow_idx == 255) begin
163         // done <= 1;
164         vstate <= VS_FINISH;
165     end else begin
166         vrow_idx <= vrow_idx + 1;
167         vstate   <= VS_WAIT;
168     end
169 end else begin
170     vcol_idx    <= vcol_idx + 1;
171     lb_rd_addr <= vcol_idx + 2; // 超前讀取
172 end
173 end
174 VS_WAIT: begin
175     // 檢查是否有足夠的新行可以進行下一次垂直濾波 (vrow_idx*2 + 5)
176     if (hrow_total >= 512 || hrow_total >= (vrow_idx << 1) + 5) begin
177         vstate <= VS_PREPARE;
178     end
179 end
180 VS_FINISH: begin
181     done    <= 1;
182     vstate <= VS_IDLE;
183 end
184 endcase
185 end
186 end

188 // output Subband Control
189 assign lv1_data      = (subband_sel == 2'b00) ? res_l1 :
190                        (subband_sel == 2'b01) ? res_h1 :
191                        (subband_sel == 2'b10) ? res_lh : res_hh;
192
193 assign lv1_out_valid = v_out_valid_l;
194 assign lv1_subband   = subband_sel;
195 assign lv1_row       = vrow_idx;
196 assign lv1_col       = vcol_idx;
197
198 endmodule

```

```

1  `timescale 1ns/1ps
2
3  module tb_dwt2d;
4
5      // parameters
6      localparam CLK_PERIOD = 10;
7      localparam IMG_SIZE   = 512;
8      localparam LV3_SIZE   = 64;
9
10     // DWT Signals
11     reg      clk, rst_n;
12     reg [7:0] pixel_in;
13     reg      pixel_valid, row_start;
14     wire     done;
15     wire     lv1_out_valid;
16     wire [1:0] lv1_subband;
17     wire [8:0] lv1_row, lv1_col;
18     wire signed [9:0] lv1_data;
19     wire     lv3_out_valid;
20     wire [1:0] lv3_subband;
21     wire [5:0] lv3_row, lv3_col;
22     wire signed [11:0] lv3_data;
23
24     // DWT Instantiation
25     dwt2d_top #(.IMG_SIZE(IMG_SIZE)) dwt (
26         .clk            (clk),
27         .rst_n          (rst_n),
28         .pixel_in       (pixel_in),
29         .pixel_valid    (pixel_valid),
30         .row_start      (row_start),
31         .done           (done),
32         .lv1_out_valid  (lv1_out_valid).

```

```

33         .lv1_subband   (lv1_subband),
34         .lv1_row       (lv1_row),
35         .lv1_col       (lv1_col),
36         .lv1_data      (lv1_data),
37         .lv3_out_valid (lv3_out_valid),
38         .lv3_subband   (lv3_subband),
39         .lv3_row       (lv3_row),
40         .lv3_col       (lv3_col),
41         .lv3_data      (lv3_data)
42     );
43
44     initial clk = 0;
45     always # (CLK_PERIOD/2) clk = ~clk;
46
47     // Memory Storage
48     reg [7:0] image [0:IMG_SIZE*IMG_SIZE-1];
49     reg signed [9:0] dut_ll3 [0:LV3_SIZE-1][0:LV3_SIZE-1];
50     reg signed [9:0] gold_ll3 [0:LV3_SIZE-1][0:LV3_SIZE-1];
51
52     integer i, j;
53     integer img_fd, gold_fd;
54
55     // Task: Load image (.raw to memory)
56     task load_image;
57         integer i;
58         begin
59             img_fd = $fopen("woman.raw", "rb");
60             if (img_fd == 0) begin
61                 $display("WARNING: woman.raw not found, generating ramp pattern for testing...");
62                 for (i = 0; i < IMG_SIZE*IMG_SIZE; i = i + 1) begin
63                     image[i] = i % 256;
64                 end

```

```

65         end else begin
66             $fread(image, img_fd);
67             $fclose(img_fd);
68             $display("Successfully loaded woman.raw");
69         end
70     end
71 endtask
72
73 // Task: Load MATLAB Golden Data
74 task load_golden;
75     integer r_val;
76 begin
77     gold_fd = $fopen("C:/VLSIDSPHW2/fixed_DWT_project/golden_ll3.txt", "r");
78     if (gold_fd == 0) begin
79         $display("WARNING: golden_ll3.txt NOT found. Simulation will fail comparison.");
80     end
81
82     for (i=0; i<LV3_SIZE; i=i+1) begin
83         for (j=0; j<LV3_SIZE; j=j+1) begin
84             if ($fscanf(gold_fd, "%d", r_val) != 1) begin
85                 $display("ERROR: Unexpected EOF in golden_ll3.txt at (%0d,%0d)", i, j);
86                 $finish;
87             end
88             gold_ll3[i][j] = r_val[11:0];
89         end
90     end
91     $fclose(gold_fd);
92     $display("Golden data loaded successfully.");
93 end
94 endtask

```

```

96 // Task: Drive Pixel Stream
97 task drive_image;
98     integer row, col;
99 begin
100     wait(rst_n == 1);
101     @(posedge clk);
102     for (row=0; row<IMG_SIZE; row=row+1) begin
103         // Row start pulse
104         row_start <= 1;
105         @(posedge clk);
106         row_start <= 0;
107
108         // Transmit pixels
109         for (col=0; col<IMG_SIZE; col=col+1) begin
110             pixel_in    <= image[row*IMG_SIZE + col];
111             pixel_valid <= 1;
112             @(posedge clk);
113         end
114         pixel_valid <= 0;
115
116         // Gap for horizontal pipeline processing
117         repeat(10) @(posedge clk);
118     end
119 end
120 endtask
121
122 // Collect outputs
123 always @(posedge clk) begin
124     if (lv3_out_valid && lv3_subband == 2'b00) begin
125         // 防止 index 越界
126         if (lv3_row < LV3_SIZE && lv3_col < LV3_SIZE)
127             dut.ll3[lv3_row][lv3_col] <= lv3_data;

```



```

128     end
129 end
130
131 // Main Processing
132 initial begin
133     // Setup
134     $dumpfile("dwt2d_sim.vcd");
135     $dumpvars(0, tb_dwt2d);
136
137     clk = 0;
138     rst_n = 0;
139     pixel_in = 0;
140     pixel_valid = 0;
141     row_start = 0;
142     i = 0; j = 0;
143
144     load_image;
145     load_golden;
146
147     // Reset Sequence
148     #(CLK_PERIOD * 10);
149     rst_n = 1;
150     #(CLK_PERIOD * 5);
151
152     $display("--- Starting DWT 2D Processing ---");
153     drive_image;
154
155     // Wait for vertical engine to finish
156     fork
157         begin : wait_done
158             wait(done == 1);
159             repeat(100) @(posedge clk);
160
161             $display("--- DWT Reported DONE ---");
162         end
163         begin : timeout
164             repeat(IMG_SIZE * IMG_SIZE * 5) @(posedge clk);
165             $display("ERROR: Simulation TIMEOUT!");
166             $finish;
167         end
168     join
169
170     compare_results;
171
172     $display("Simulation Finished at %0t", $time);
173     $finish;
174 end
175
176 // Comparison Task
177 task compare_results;
178     integer errs;
179     begin
180         errs = 0;
181         for (i=0; i<LV3_SIZE; i=i+1) begin
182             for (j=0; j<LV3_SIZE; j=j+1) begin
183                 if (dut_ll3[i][j] != gold_ll3[i][j]) begin
184                     if (errs < 20)
185                         $display("Mismatch found at (%0d,%0d): %0d vs %0d", i, j, dut_ll3[i][j], gold_ll3[i][j]);
186                     errs++;
187                 end
188             end
189         end
190
191         $display("***** FAILURE: %0d mismatches found. *****", errs);
192     end
193 endtask
194
195 endmodule

```